

Semana 8: Criptografia Assimétrica

DH (Diffie Hellman)

Relembre o protocolo de acordo de chaves *Diffie_Hellman*:

1. Alice $\rightarrow$ Bob : g^x
2. Bob $\rightarrow$ Alice : g^y
3. Alice, Bob : $K = g^{(x*y)}$

Onde g e p são os parâmetros públicos do protocolo; (x, g^x) e (y, g^y) são os pares de chaves de Alice e Bob respectivamente; e K é o segredo estabelecido pelo protocolo.

Considere o seguinte template de código Python que simula localmente o protocolo *Diffie_Hellman*.

```
from multiprocessing import Process, Pipe

def alice_process(conn):
    # Generate keys ...
    # 1. Alice  $\rightarrow$  Bob
    conn.send(alice_bytes)
    # 2. Bob  $\rightarrow$  Alice
    bob_bytes = conn.recv()
    # 3. Alice, Bob ...

def bob_process(conn):
    # Generate keys ...
    # 1. Alice  $\rightarrow$  Bob
    alice_bytes = conn.recv()
    # 2. Bob  $\rightarrow$  Alice
    conn.send(bob_bytes)
    # 3. Alice, Bob ...

if __name__ == '__main__':
    parent_conn, child_conn = Pipe()
    p1 = Process(target=alice_process, args=(parent_conn,))
    p2 = Process(target=bob_process, args=(child_conn,))
    p1.start(); p2.start()
    p1.join(); p2.join()
```

Note que, embora os dois processos para Alice e Bob sejam simulados, a informação que trocam deve ser em `bytes`.

PROG: `dh.py`

Pretende-se adaptar os programas realizados no ponto anterior para que a chave por eles utilizada resulte da execução do protocolo de acordo de chaves *Diffie-Hellman*. Para isso iremos fazer uso da funcionalidade oferecida pela biblioteca `cryptography`, utilizando a classe `dh`, que acabará por "esconder" a matemática apresentada acima. Em cada processo, imprima a chave K no ecrã para validar que é efetivamente a mesma para Alice e Bob.

Algumas observações:

- Para gerar chaves *Diffie Hellman*, temos primeiro de criar um objeto `DHParameters`. Este objeto é que dispõe do método que permite gerar a chave privada, da qual se poderá obter a chave pública respectiva.
- Pode gerar os parâmetros recorrendo ao método `dh.generate_parameters` – note que este processo pode demorar algum tempo.
 - fixar os parâmetros (p, g) , em que p é o primo que define o corpo e g o gerador adoptado. Pode usar valores fixos para estes parâmetros, e.g.:

```

p = 0x
FFFFFFFF FFFFFFFF C90FDAA2 2168C234 C4C6628B
80DC1CD1 29024E08 8A67CC74 020BBEA6 3B139B22
514A0879 8E3404DD EF9519B3 CD3A431B 302B0A6D
F25F1437 4FE1356D 6D51C245 E485B576 625E7EC6
F44C42E9 A637ED6B 0BFF5CB6 F406B7ED EE386BFB
5A899FA5 AE9F2411 7C4B1FE6 49286651 ECE45B3D
C2007CB8 A163BF05 98DA4836 1C55D39A 69163FA8
FD24CF5F 83655D23 DCA3AD96 1C62F356 208552BB
9ED52907 7096966D 670C354E 4ABC9804 F1746C08
CA18217C 32905E46 2E36CE3B E39E772C 180E8603
9B2783A2 EC07A28F B5C55DF0 6F4C52C9 DE2BCBF6
95581718 3995497C EA956AE5 15D22618 98FA0510
15728E5A 8AACAA68 FFFFFFFF FFFFFFFF

g = 2

```

- Em criptografia assimétrica, as chaves têm uma estrutura complexa e não são meros *arrays* de *bytes* como era o caso das chaves simétricas usadas até agora. Torna-se assim necessário **serializar** essas chaves (que são objetos de classes *Python*) antes de as comunicar pela rede. Podem consultar a documentação relevante [aqui](#). Podem escolher entre os *encodings* `DER` ou `PEM`, sendo que a única distinção prática é que `DER` é um formato binário e `PEM` usa caracteres imprimíveis semelhante ao *base64*.

PROG: `dh_aes_gcm.py`

Estenda o programa `dh.py` anterior para, depois de efetuar a troca de chaves, trocar adicionalmente uma mensagem confidencial entre Alice e Bob. Para isso, utilize a cifra `AES-GCM` explorada no guião anterior.

Algumas observações:

- Note que no DH (e em criptografia assimétrica no geral) a chave `K` é um grande número com dadas propriedades e raramente pode/deve ser utilizada diretamente como segredo criptográfico, e.g. como chave de uma cifra simétrica. Deve-se portanto recorrer a uma KDF (e.g. `HKDF`, conforme sugestão apresentada na documentação).

QUESTÃO: Q1

Assumindo que trocava não uma mas várias mensagens, utilizando a KDF para o efeito, a aplicação desenvolvida garante *PFS* (*Perfect Forward Secrecy*), i.e. a segurança das comunicações não é comprometida mesmo que o segredo `K` seja comprometido? Justifique.

Certificados

Relembre que o protocolo DH é completamente vulnerável a ataques de *man-in-the-middle*. Para evitar tais ataques, o recurso a criptografia assimétrica pressupõe tipicamente a utilização de certificados que estabeleçam a autenticidade das chaves públicas utilizadas. Vamos por isso estender o programa que tem vindo a ser construído com certificados [X509](#).

Os certificados que iremos utilizar irão conter chaves RSA. Crie chaves secretas para a CA (autoridade de certificação), a Alice e o Bob:

```

openssl genrsa -out CA.key 2048
openssl genrsa -out Alice.key 2048
openssl genrsa -out Bob.key 2048

```

De seguida, crie os certificados para cada um:

```

openssl req -x509 -new -nodes -key CA.key -sha256 -days 365 -out CA.crt -subj "/CN=CA"
openssl req -new -key Alice.key -out Alice.csr -subj "/CN=Alice"
openssl x509 -req -in Alice.csr -CA CA.crt -CAkey CA.key -CAcreateserial -out Alice.crt -days 365 -sha256
openssl req -new -key Bob.key -out Bob.csr -subj "/CN=Bob"
openssl x509 -req -in Bob.csr -CA CA.crt -CAkey CA.key -CAcreateserial -out Bob.crt -days 365 -sha256

```

Note que o certificado da CA é self-signed, e os certificados de Alice e Bob são assinados pela CA. O procedimento tem duas etapas: criar primeiro um CSR (Certificate Signing Request) com a chave secreta do utilizador, que é depois assinado pela CA.

QUESTÃO: Q2

Onde está armazenada a informação das chaves públicas de cada participante?

DH Autenticado (protocolo *Station-to-Station*)

Por último, pretende-se complementar o acordo de chaves *Diffie-Hellman* para incluir a funcionalidade para garantir autenticidade dos participantes. Recorde que no protocolo *Station-to-Station* é adicionada uma troca de assinaturas:

- Alice $\rightarrow$ Bob : g^x
- Bob $\rightarrow$ Alice : $g^y, \text{Sig}_B(g^y, g^x), \text{Cert}_B$
- Alice $\rightarrow$ Bob : $\text{Sig}_A(g^x, g^y), \text{Cert}_A$

4. Alice, Bob : $K = g^{(x*y)}$

PROG: `sts_aes_gcm.py`

Adapte o programa `dh_aes_gcm.py` para substituir o *Diffie-Hellman* tradicional pelo protocolo *Station-to-Station*.

Algumas observações:

- O algoritmo de assinatura que iremos utilizar é o [RSA](#), que pressupõe a utilização de um mecanismo de *padding* (e.g. PSS). Esse *padding* tem uma "natureza" diferente do *padding* simétrico usado noutros guiões.
- As chaves a utilizar no protocolo *Station-to-Station* são as de Alice e Bob. Ambos devem utilizar os certificados um do outro para validar as assinaturas trocadas. Devem também utilizar o certificado da CA para validar os certificados trocados.
- Uma possível dificuldade neste guião resulta de gerir a troca de mensagens envolvendo várias componentes cujos tamanhos não são fáceis de prever. Para isso, sugere-se que utilizem as funções apresentadas abaixo, que incluem informação dos tamanhos na **serialização** de um par de *bytestrings*.

```
def mkpair(x, y):
    """produz uma byte-string contendo o tuplo '(x,y)' ('x' e 'y' são byte-strings)"""
    len_x = len(x)
    len_x_bytes = len_x.to_bytes(2, "little")
    return len_x_bytes + x + y

def unpair(xy):
    """extraí componentes de um par codificado com 'mkpair'"""
    len_x = int.from_bytes(xy[:2], "little")
    x = xy[2 : len_x + 2]
    y = xy[len_x + 2 :]
    return x, y
```

Note que agora a função `unpair` recupera cada componente do par sem necessitar de se passar informação de tamanhos (e.g. `unpair(mkpair(b'abcde',b'99ijjhh')) = (b'abcde', b'99ijjhh')`).

QUESTÃO: Q3

Se a Alice omitir o passo de verificação do certificado do Bob (utilizando a CA), mas mantiver a verificação da assinatura do Bob prevista no protocolo *Station-to-Station*, o protocolo continua a ser imune a ataques de Man-in-the-Middle?